Name: [Student Name, e.g., Alice Smith] | Subject: Data Structures and Algorithms | Date: May 24, 2025

Section A: Descriptive Questions

Q1. Define Stack. Write its applications.

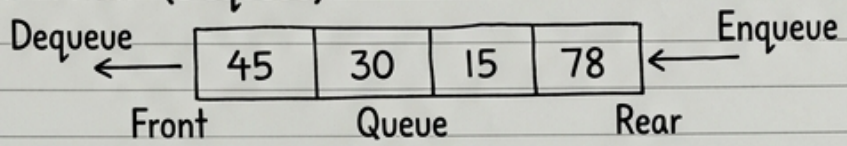
A stack is a linear data structure that follows the LIFO (Last-In-First-Out) principle. This means the last element added is the first one removed.

Applications include:

1. Function Call Management: Used by compilers for recursion.
2. Expression Evaluation: In parsing and evaluating mathematical expressions.
3. Undo/Redo Mechanisms: In text editors.

Q2. Explain the working of Queue with a diagram.

A queue is a linear data structure that follows the FIFO (First-In-First-Out) principle. Elements are inserted at the Rear (Enqueue) and removed from the Front (Dequeue).



Q3. What is the time complexity of Binary Search? Explain.

The average and worst-case time complexity of Binary Search is $O(\log n)$. This is because in each step of the algorithm, the search space (the number of elements to consider) is reduced by half. If the target is not at the middle, the search continues in either the left or right sub-array. The number of times 'n' can be divided by 2 to reach 1 is roughly $\log_2 n$.

Q4. Write short notes on Tree Traversal.

Tree Traversal refers to the process of visiting each node in a tree data structure exactly once. There are three common types of Depth-First traversal:

1. In-order (Left, Root, Right): Visits in ascending order for BST.
2. Pre-order (Root, Left, Right): Useful for copying trees.
3. Post-order (Left, Right, Root): Useful for deleting trees.

There is also Breadth-First traversal (Level-Order).

Q5. Given a linked list, explain how insertion at the beginning is performed.

To insert a new node at the beginning of a singly linked list:

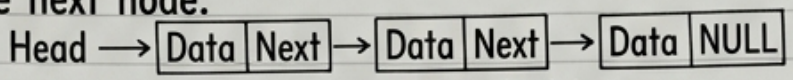
1. Create a new node with the required data.
2. Set the next pointer of the new node to point to the current Head of the list.
3. Update the Head pointer to point to the new node.

Before: [NewNode] → [Node1] → ... After: [Head] → [NewNode] → [Node1] → ...

Assume standard data.

Name: [Student Name, e.g., Alice Smith] | Subject: Data Structures and Algorithms | Date: May 24, 2025

Q6. Explain Linked List and write its types with a diagram.
A Linked List is a linear data structure where elements are not stored in contiguous memory. Each element (node) contains data and a pointer to the next node.



Singly Linked List

Types include:
1. Singly Linked List (pictured)
2. Doubly Linked List (nodes have prev and next pointers)
3. Circular Linked List

Q7. What is Hashing? Explain with an example. [Note: This is an interyption.]
Hashing is a way to encrypt data to keep it secure. It takes a piece of text and scrambles it into a code that only the correct person can decrypt with a password. This is crucial for keeping user passwords safe in a database.
Example: For a user 'password123', the system would apply a hashing encryption to get something like 'x7#8jK9@', and only the system can turn that code back into 'password123' to check it.

Q8. Explain the working of Stack using an example.
A Stack works on the LIFO principle (Last-In-First-Out), like a stack of plates. The only allowed operations are Push (add) and Pop (remove) from the top.
Example: To model an undo function in an editor.
Push(State 1) → Stack: [State 1]
Push(State 2) → Stack: [State 1, State 2]
Push(State 3) → Stack: [State 1, State 2, State 3]
Undo (Pop()) → Returns State 3. Stack is now [State 1, State 2].

Q9. What is Recursion? Write a recursive function to calculate factorial of a number.
Recursion is when a function calls itself, directly or indirectly. It must have a base case to terminate and a recursive step to progress toward the base case.
Function in pseudo-code:

```
int factorial(int n) {
    if (n <= 1) { return 1;            // Base case
    } else {
        return n * factorial(n - 1);   // Recursive step
}
```

Q10. Differentiate between Array and Linked List.
An Array is a dynamic data structure where elements are connected by pointers, making insertion and deletion from any position fast (O(1)). Size is flexible and grows as needed, but it has slow access time (O(n)).
A Linked List, on the other hand, stores elements in a continuous block of memory. This allows for very fast random access (O(1)), but making changes to the size (insertion or deletion) is expensive and slow, needing O(n) time.